クリックスルー率予測データの処理とモデルトレーニング

NetApp Solutions

NetApp
January 13, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/ai/aks-anf_libraries_for_data_processing_and_model_training.html on January 13, 2022. Always check docs.netapp.com for the latest.

目次

クリックスルー率予測データの処理とモデルトレーニング 1
データ処理およびモデルトレーニング用のライブラリ 1
Pandas で Logs Day 15 をクリックして、 scikit に学習したランダムフォレストモデルをトレーニングします 1
Dask の 15 日目をロードし、 Dask cuML ランダムフォレストモデルをトレーニングします 3
ネイティブタスクストリームダッシュボードを使用して Dask を監視します 5
トレーニング時間の比較 6
Prometheus と Grafana で Dask と Rapids を監視します 7
NetApp DataOps ツールキットを使用したデータセットとモデルのバージョン管理 7
Jupyter ノートブックを参考にしてください 7

クリックスルー率予測データの処理とモデルトレーニング

データ処理およびモデルトレーニング用のライブラリ

"前のバージョン： Azure NetApp Files のパフォーマンス階層"

次の表に、このタスクの構築に使用されたライブラリとフレームワークを示します。これらのコンポーネントはすべて、 Azure の役割ベースのアクセスおよびセキュリティ制御と完全に統合されています。

ライブラリ / フレームワーク	説明
Dask cuML	ML を GPU で動作させるには、を使用します "cuML ライブラリ" Dask を使用して Rapids cuML パッケージにアクセスできます。Rapids cuML は、クラスタリング、寸法縮小、回帰アプローチなどの一般的な ML アルゴリズムを高性能 GPU ベースの実装で実装し、 CPU ベースのアプローチで最大 100 倍のスピードアップを実現します。
Dask cuDF	cuDF には、データのサブ設定、変換、ワンホットエンコーディングなど、 GPU アクセラレーションによる抽出、変換、読み込み（ ETL ）をサポートするその他のさまざまな機能があります。Rapids チームはを維持する "dask -cudf ライブラリ" これには、 Dask および cuDF を使用するためのヘルパーメソッドが含まれています。
Scikit learn	Scikit-Learn には、数十の機械学習アルゴリズムとモデルが組み込まれています。これらは、試算ツールと呼ばれます。各 "エスティメータ" は、を使用して一部のデータに装着できます メソッド

2 つのノートブックを使用して、比較のための ML パイプラインを構築しました。 1 つは従来の Pandas の坐骨坐骨学習アプローチで、もう 1 つは Rapids および Dask との分散トレーニングです。各ノートブックを個別にテストして、パフォーマンスを時間と規模の観点から確認できます。各ノートブックについて個別に説明し、 Rapids および Dask を使用した分散型トレーニングの利点を示します。

"次に、 Pandas で Logs Day 15 をクリックし、 scikit に学習したランダムな森林モデルをトレーニングします。"

Pandas で Logs Day 15 をクリックして、 scikit に学習したランダムフォレストモデルをトレーニングします

"Previous ：データ処理とモデルトレーニング用のライブラリ。"

このセクションでは、 Pandas と Dask DataFrames を使用して、 Criteo Terabyte データセットから Click Logs データをロードする方法について説明します。このユースケースは、広告交換のためのデジタル広告において、広告がクリックされるかどうかを予測したり、交換品が自動化されたパイプラインで正確なモデルを使用していないかどうかを予測したりすることで、ユーザーのプロファイルを作成する場合に適しています。

Click Logs データセットから 15 日目のデータをロードし、合計 45GB にしました。Jupyter ノートブックの ctr-pandasrf-colated で次のセルを実行すると、最初の 5,000 万行を含む Pandas DataFrame が作成され、scikit 学習ランダムフォレストモデルが生成されます。

```
%%time
import pandas as pd
import numpy as np
header = ['col'+str(i) for i in range (1,41)] #note that according to
criteo, the first column in the dataset is Click Through (CT). Consist of
40 columns
first_row_taken = 50_000_000 # use this in pd.read_csv() if your compute
resource is limited.
# total number of rows in day15 is 20B
# take 50M rows
"""
Read data & display the following metrics:
1. Total number of rows per day
2. df loading time in the cluster
3. Train a random forest model
"""
df = pd.read_csv(file, nrows=first_row_taken, delimiter='\t',
names=header)
# take numerical columns
df_sliced = df.iloc[:, 0:14]
# split data into training and Y
Y = df_sliced.pop('col1') # first column is binary (click or not)
# change df_sliced data types & fillna
df_sliced = df_sliced.astype(np.float32).fillna(0)
from sklearn.ensemble import RandomForestClassifier
# Random Forest building parameters
# n_streams = 8 # optimization
max_depth = 10
n_bins = 16
n_trees = 10
rf_model = RandomForestClassifier(max_depth=max_depth,
n_estimators=n_trees)
rf_model.fit(df_sliced, Y)
```

トレーニングされたランダムフォレストモデルを使用して予測を実行するには、このノートブックで次の段落を実行します。重複を避けるために、 15 日目から最後の 100 万行をテストセットとして使用しました。セルはまた、モデルの発生率として定義された予測精度を計算し、ユーザーが広告をクリックするかどうかを正確に予測します。このノートブックの構成部品を確認するには、を参照してください "公式の坐骨神経痛 - 学習文書"。

```
# testing data, last 1M rows in day15
test_file = '/data/day_15_test'
with open(test_file) as g:
    print(g.readline())

# dataFrame processing for test data
test_df = pd.read_csv(test_file, delimiter='\t', names=header)
test_df_sliced = test_df.iloc[:, 0:14]
test_Y = test_df_sliced.pop('col1')
test_df_sliced = test_df_sliced.astype(np.float32).fillna(0)
# prediction & calculating error
pred_df = rf_model.predict(test_df_sliced)
from sklearn import metrics
# Model Accuracy
print("Accuracy:",metrics.accuracy_score(test_Y, pred_df))
```

"次は、 Dask の 15 日目をロードし、 Dask cuML ランダムフォレストモデルをトレーニングします。"

Dask の 15 日目をロードし、 Dask cuML ランダムフォレストモデルをトレーニングします

"前の手順： Pandas で Logs Day 15 をクリックし、 scikit に学習したランダムなフォレストモデルをトレーニングします。"

前のセクションと同様に、 Pandas の Logs Day 15 をロードして、 scikit に学習したランダムフォレストモデルをトレーニングします。この例では、 Dask cuDF を使用して DataFrame のロードを実行し、 Dask cuML でランダムなフォレストモデルのトレーニングを行いました。セクションのトレーニング時間と規模の違いを比較しました "「トレーニング時間の比較」"

Crito_dASK_RF.ipynb

このノートブックは、次の例に示すように、「 numpy`, cuml` 」、および必要な「 Ask 」ライブラリをインポートします。

```
import cuml
from dask.distributed import Client, progress, wait
import dask_cudf
import numpy as np
import cudf
from cuml.dask.ensemble import RandomForestClassifier as cumlDaskRF
from cuml.dask.common import utils as dask_utils
```

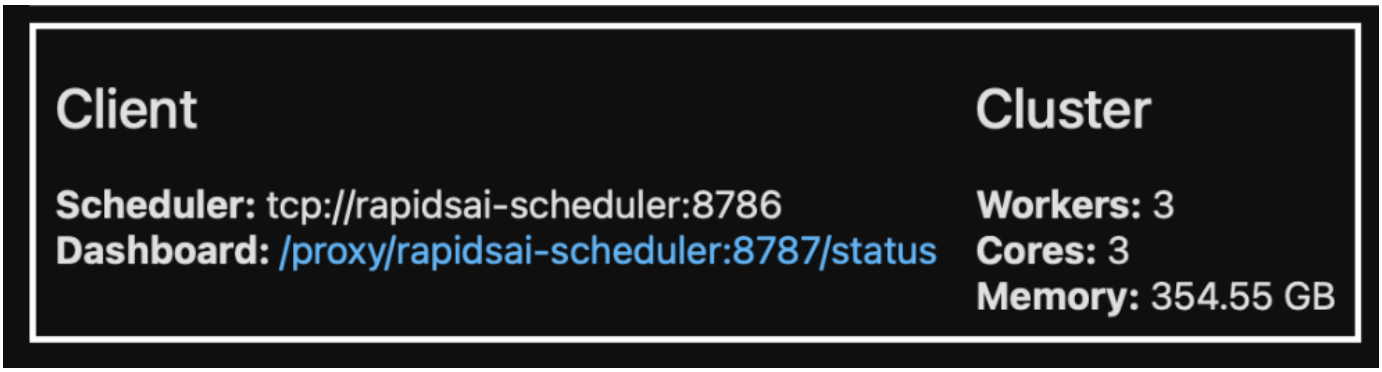
Dask Client() を開始します。

```
client = Client()
```

クラスタが正しく設定されていれば、ワーカーノードのステータスを確認できます。

```
client
workers = client.has_what().keys()
n_workers = len(workers)
n_streams = 8 # Performance optimization
```

AKS クラスタでは、次のステータスが表示されます。



Dask は遅延実行パラダイムを採用しています。処理コードを瞬時に実行するのではなく、 Dask は Directed Acyclic Graph （ DAG ）を実行します。DAG には、各ワーカーが実行する必要のある一連のタスクとそのやり取りが含まれています。このレイアウトは、ユーザーが Dask に一方の方法または別の方法で実行するように指示するまで、タスクが実行されないことを意味します。Dask を使用すると、 3 つの主なオプションがあります。

- * DataFrame 上のコールコンピュート () 。 * このコールはすべてのパーティションを処理し、結果をスケジューラに返して最終的な集約を行い、 cuDF DataFrame に変換します。このオプションは慎重に使用してください。スケジューラノードのメモリが不足しないかぎり、結果が大幅に低下する場合にのみ使用してください。
- * DataFrame 上で Persist() を呼び出します。 * この呼び出しはグラフを実行しますが、スケジューラノードに結果を返すのではなく、クラスタ内で結果をメモリに保持するため、ユーザは同じ処理を再実行することなくパイプライン内で中間結果を再利用できます。
- * DataFrame 上のコールヘッド () 。 * cuDF と同様に、この呼び出しは 10 件のレコードをスケジューラノードに返します。このオプションを使用すると、 DataFrame に目的の出力形式が含まれているかどうか、または処理と計算に応じてレコード自体が適切かどうかをすばやく確認できます。

したがって、ユーザがこれらのアクションのいずれかをコールしない限り、スケジューラが処理を開始するのを待機するアイドル状態になります。この遅延実行パラダイムは、 Apache Spark などの最新の並列および分散コンピューティングフレームワークで一般的です。

次の段落では、 Dask cuML を使用して、 GPU アクセラレーションによる分散コンピューティングを行い、モデル予測精度を計算することにより、ランダムなフォレストモデルのトレーニングを行います。

```
Adsf
# Random Forest building parameters
n_streams = 8 # optimization
max_depth = 10
n_bins = 16
n_trees = 10
cuml_model = cumlDaskRF(max_depth=max_depth, n_estimators=n_trees,
n_bins=n_bins, n_streams=n_streams, verbose=True, client=client)
cuml_model.fit(gdf_sliced_small, Y)
# Model prediction
pred_df = cuml_model.predict(gdf_test)
# calculate accuracy
cu_score = cuml.metrics.accuracy_score( test_y, pred_df )
```

"次の手順：ネイティブタスクストリームダッシュボードを使用して Dask を監視します。"

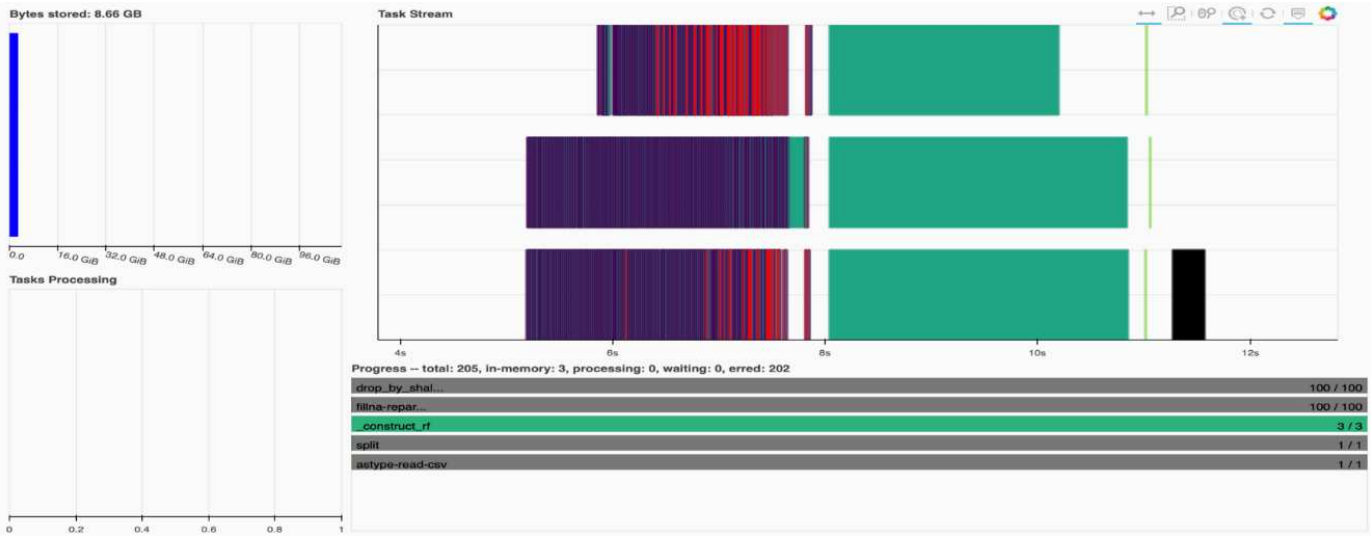
ネイティブタスクストリームダッシュボードを使用して Dask を監視します

"前のページ : Dask の 15 日目をロードし、 Dask cuML ランダムフォレストモデルをトレーニングします。"

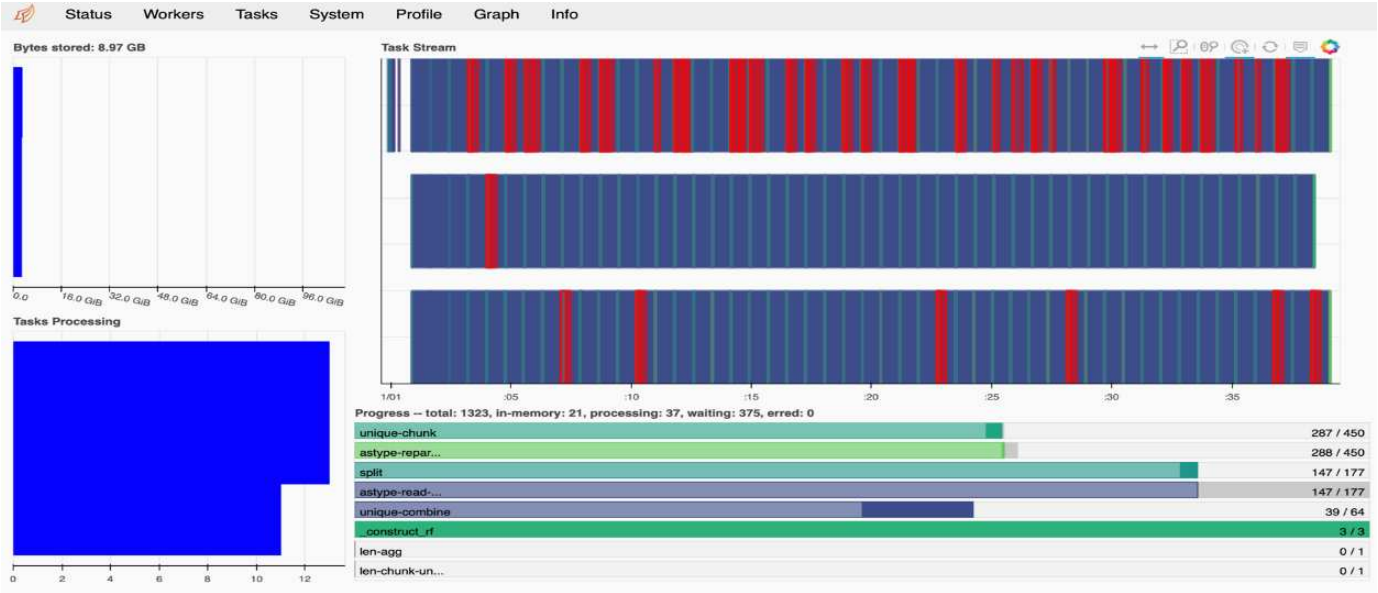
。 "Dask 分散スケジューラ" ライブフィードバックは、次の 2 つの形式で提供します。

- ライブ情報を含む多数のプロットやテーブルを含むインタラクティブなダッシュボード
- コンソールやノートブックでの対話型の使用に適したプログレスバーです

この場合、次の図は、保存されたバイト数、ストリーム数の詳細な内訳を示すタスクストリーム、実行された関連機能を持つタスク名ごとの進捗状況を監視する方法を示しています。この例では、ワーカーノードが 3 つあるため、ストリームには 3 つの主要なチャンクがあり、各ストリーム内で異なるタスクを示すカラーコードがあります。



個々のタスクを分析し、実行時間をミリ秒単位で調査するか、障害や障害を特定することができます。たとえば、次の図は、ランダムフォレストモデルフィッティングステージのタスクストリームを示しています。実行される関数は、 DataFrame 処理用の一意のチャンク、ランダムフォレストをフィッティングするための _construct_RF など、はるかに多くあります。Criteo のクリックログに含まれる 1 日分のデータのサイズ（ 45GB ）が大きいため、 DataFrame の処理にほとんどの時間が費やされていました。



"次：トレーニング時間の比較。"

トレーニング時間の比較

"前の手順：ネイティブタスクストリームダッシュボードを使用して Dask を監視します。"

このセクションでは、従来の Pandas を使用したモデルのトレーニング時間を Dask と比較します。Pandas では、メモリオーバーフローを回避するために、処理時間が遅くなるため、より少量のデータをロードしました。そのため、結果を補間して公平な比較を行いました。

次の表は、 Pandas ランダムフォレストモデルに使用されるデータが大幅に少ない場合の、生のトレーニング時間の比較を示しています (データセットの 1 日あたりの 2000 億行のうち、 5,000 万行) 。このサンプルでは、使用可能なすべてのデータの 0.25% 未満しか使用されていません。DASK cuML の場合は '20 億行すべての使用可能なローについてランダムフォレストモデルをトレーニングしましたこの 2 つのアプローチでは、同等のトレーニング時間が得られました

アプローチ	トレーニング時間
Scikit - Learn ：トレーニングデータとして day15 の 50 M 行のみを使用します	47 分 21 秒
Rapids-DASK ：トレーニングデータとして、 Day15 のすべての 20B 行を使用します	1 時間 12 分 11 秒

次の表に示すように、トレーニング時間の結果を直線的に補間する場合、 Dask を使用した分散型トレーニングを使用すると大きな利点があります。従来の Pandas の scikit 学習アプローチでは、クリックログ 1 日あたり 45 GB のデータを処理してトレーニングするのに 13 日かかりますが、 Rapids-Dask アプローチでは同じ量のデータを処理するのにかかる時間は 262.39 倍になります。

アプローチ	トレーニング時間
Scikit - Learn ：トレーニングデータとして day15 のすべての 20B 行を使用します	13 日、 3 時間、 40 分、 11 秒
Rapids-DASK ：トレーニングデータとして、 Day15 のすべての 20B 行を使用します	1 時間 12 分 11 秒

前の表では、 Dask と Rapids を使用してデータ処理とモデルトレーニングを複数の GPU インスタンスに分散することで、従来の Pandas DataFrame 処理と比較して、 scikit 学習モデルトレーニングでの実行時間が大幅に短縮されたことを確認できます。このフレームワークを使用すると、マルチノードのマルチ GPU クラスタ内だけでなく、クラウド内でもオンプレミスでのスケールアップとスケールアウトが可能です。

"次の例： Prometheus と Grafana で Dask と Rapids を監視します。"

Prometheus と Grafana で Dask と Rapids を監視します

"前：トレーニング時間の比較。"

すべてのデータを導入したら、新しいデータに対して推論を実行します。このモデルは、ユーザーが閲覧アクティビティに基づいて広告をクリックするかどうかを予測します。予測の結果は Dask cuDF に格納されます。Prometheus で結果を監視し、 Grafana ダッシュボードで視覚化できます。

詳細については、を参照してください "Rapids AI 培地ポスト"。

"次のセクション： NetApp DataOps ツールキットを使用したデータセットとモデルのバージョニング"

NetApp DataOps ツールキットを使用したデータセットとモデルのバージョン管理

"以前： Prometheus と Grafana を使用して Dask と Rapids を監視していました。"

NetApp DataOps Toolkit for Kubernetes は、ストレージリソースと Kubernetes ワークロードをデータサイエンスのワークスペースレベルまで抽象化します。これらの機能は、データサイエンティストとデータエンジニア向けに設計された、使いやすいシンプルなインターフェイスにパッケージ化されています。使い慣れた Python プログラムを使用しており、データサイエンティストやエンジニアは JupyterLab ワークスペースをわずか数秒でプロビジョニングおよび削除できます。これらのワークスペースには、テラバイト、あるいはペタバイト規模のストレージ容量が含まれることがあり、データサイエンティストは、すべてのトレーニングデータセットをプロジェクトのワークスペースに直接格納できます。ワークスペースとデータボリュームを個別に管理する時代は終わりました。

詳細については、ツールキットを参照してください "GitHub リポジトリ"。

"次は終わりです"

Jupyter ノートブックを参考にしてください

"前： NetApp DataOps ツールキットを使用したデータセットとモデルのバージョニング。"

このテクニカルレポートには、 Jupyter ノートブックが 2 つ関連付けられています。

- "*CTR - PandasRF - 照合済み。 ipynb. *" このノートブックは Crito Terabyte Logs データセットから 15 日目を読み込み、データを Pandas DataFrame に処理してフォーマットし、 Scikit-learn ランダムフォレストモデルのトレーニングを行い、予測を実行し、精度を計算します。
- "* Crito_dAsk _RF.ipynb.*" このノートブックは Crito Terabyte Logs データセットから 15 日目をロードし、データを Dask cuDF に処理してフォーマットし、 Dask cuML ランダムフォレストモデルのトレーニングを行い、予測を実行し、精度を計算します。GPU を搭載した複数のワーカーノードを活用することで、この分散データとモデルの処理とトレーニングのアプローチを非常に効率的に行うことができます。処理するデータが多いほど、従来の ML アプローチに比べて時間を大幅に節約できます。このノートブックは、ネットワークセットアップによってデータやモデルの配布が自由に移動できる限り、クラウド、オンプレミス、または Kubernetes クラスタにコンピューティングとストレージが異なる場所に配置されているハイブリッド環境に導入できます。

"次は終わりです"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.